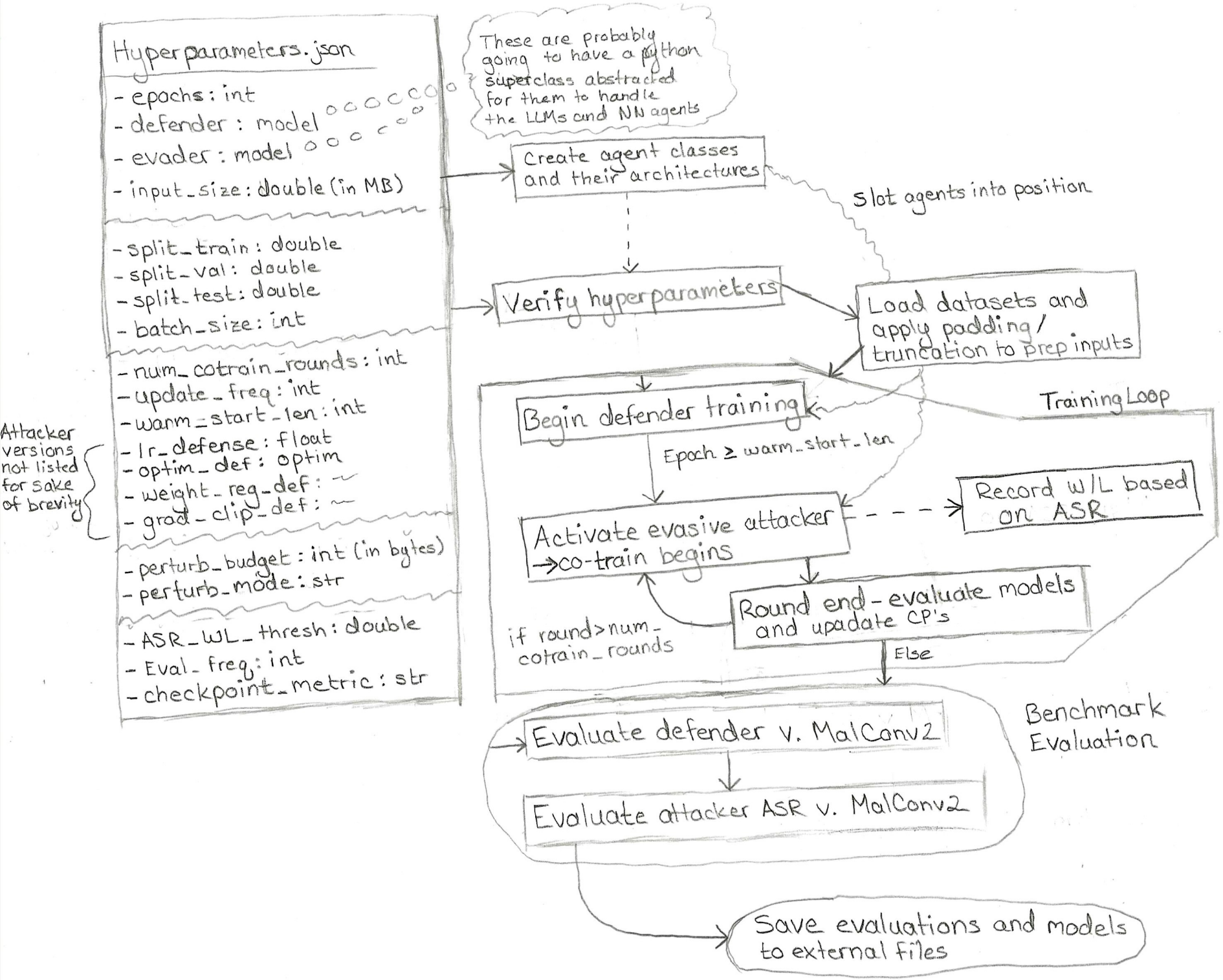
Hyperparameters.json
- epochs : int
- defender : model
- evader : model
- input_size : double (in MB)
- split_train : double
- split_val : double
- split_test : double
- batch_size : int
- num_cotrain_rounds : int
- update_freq : int
- warm_start_len : int
- lr_defense : float
- optim_def : optim
- weight_reg_def : ~
- grad_clip_def : ~
- perturb_budget : int (in bytes)
- perturb_mode : str
- ASR_WL_thresh : double
- Eval_freq : int
- checkpoint_metric : str
Attacker versions not listed for sake of brevity
These are probably going to have a python superclass abstracted for them to handle the LLMs and NN agents
Create agent classes and their architectures
Slot agents into position
Verify hyperparameters
Load datasets and apply padding/truncation to prep inputs
Training Loop
Begin defender training
Epoch ≥ warm_start_len
Activate evasive attacker →co-train begins
Record W/L based on ASR
Round end - evaluate models and upadate CP's
if round > num_cotrain_rounds
Else
Benchmark Evaluation
Evaluate defender v. MalConv2
Evaluate attacker ASR v. MalConv2
Save evaluations and models to external files